Entropy-Regularized Rewards for Self-Rewarded Training: Delaying Reward Hacking in Majority-Vote RL for Mathematical Reasoning

Daniel Shavit
Department of Industrial Engineering and Management, Ben Gurion University
shavda@bgu.ac.il

Yuval Zohar
Department of Industrial Engineering and Management, Ben Gurion University
yuvalzoh@post.bgu.ac.il

Abstract

Self-Rewarded Training (SRT) (Shafayat et al., 2025) trains large reasoning models on mathematical problem-solving without ground-truth labels, using majority-vote agreement across sampled rollouts as a proxy reward. SRT improves accuracy early in training, but the original paper shows that extended training causes the proxy reward to be corrupted: the model learns to maximize self-agreement without becoming more correct, and performance collapses. We propose adding a per-rollout entropy (surprisal) bonus to the SRT reward – a classical exploration-preserving regularizer from policy-gradient RL (Williams and Peng, 1991; Mnih et al., 2016; Haarnoja et al., 2018) – to test whether it delays or reduces this collapse. We implement a lightweight, dependency-minimal replication of SRT's RLOO training loop on Qwen2.5-Math-1.5B over the DAPO dataset (Yu et al., 2025), and compare four reward conditions – plain SRT (α=0), two entropy-bonus strengths (α=0.3, α=0.5), and a ground-truth ceiling condition – each trained for 900 steps with periodic held-out evaluation. On the final, low-noise 100-question evaluation, plain SRT finished training with a clearly positive agreement-accuracy gap ($+0.043$), the reward-hacking signature the original paper describes; a moderate entropy bonus (α=0.3) achieved the best result of the three trained conditions (highest accuracy, smallest gap), while a stronger bonus (α=0.5) underperformed both alternatives – a non-monotonic effect of α. Surprisingly, the ground-truth condition did not behave like a ceiling at this training budget: its final accuracy (0.25) was on par with or below all three label-free conditions, suggesting that reward *density* (how often a rollout receives useful signal) may matter as much as reward *correctness* within a fixed, modest training budget. A preliminary frozen-model pilot across three off-the-shelf models confirms that the agreement-accuracy gap SRT relies on is a real, measurable phenomenon prior to any training, motivating the intervention studied here.

1 Introduction

Reinforcement learning has become the dominant recipe for turning a pretrained language model into a strong mathematical reasoner, but almost every successful recipe – from RLHF to verifiable-reward RL for math – depends on a reward signal that requires either human-labeled preferences or a

Preprint.

ground-truth answer key. Both are expensive to obtain at scale, which motivates *label-free* RL: can a model improve at reasoning using only signals it can compute about its own outputs, with no external verifier?

Shafayat et al. (2025) study exactly this question with **Self-Rewarded Training (SRT)**. At every RL step, SRT samples several candidate solutions to the same problem from the *current* policy, takes their majority-vote answer as a stand-in for ground truth, and rewards each rollout for agreeing with that majority. This builds directly on self-consistency decoding (Wang et al., 2022), which showed that majority voting over multiple chain-of-thought samples is itself a strong test-time proxy for correctness. SRT turns that proxy into a training signal: no labels are needed because the model is, in effect, both "the student" and "the teacher".

This works, at first. Shafayat et al. report genuine accuracy gains with no ground-truth supervision at all. But they also identify the method's central failure mode, which motivates this project: with **prolonged training**, the policy can learn to maximize the self-consistency reward without becoming more *correct* – it collapses onto confidently repeating one answer per question, right or wrong, because that guarantees perfect agreement with itself. The self-consistency reward is maximized precisely as true accuracy stagnates or falls. This is a textbook instance of reward hacking: a proxy stops being a good measure once it is directly optimized, and the original paper leaves *fixing* it as an open problem rather than a solved one.

Our contribution. We propose a direct, mechanistically-motivated fix: augment SRT's reward with a small bonus proportional to how *surprising* (rare) a rollout's answer is relative to the other sampled rollouts for that question. This is an application of entropy regularization – a decades-old technique for preventing premature convergence in policy-gradient RL (Williams and Peng, 1991), popularized by A3C (Mnih et al., 2016) and formalized as an explicit training objective by maximum-entropy RL methods such as Soft Actor-Critic (Haarnoja et al., 2018) – applied specifically to SRT's self-consistency collapse, which the original paper does not attempt to address. We implement this as a lightweight, from-scratch RLOO (Kool et al., 2019; Ahmadian et al., 2024) training loop, add a ground-truth condition as an upper-bound reference the original paper does not report, and run a controlled comparison across four reward conditions at 900 training steps each – chosen because reward hacking in the original paper's own experiments only becomes visible after substantial training.

The rest of this report is organized as follows. Section 2 reviews the RL background needed to follow SRT and our modification. Section 3 states our reward function and hypotheses precisely. Section 4 describes the experimental setup. Section 5 reports results across all four 900-step conditions. Section 6 discusses limitations and future work.

2 Background

2.1 Policy-gradient RL and RLOO

We treat generating a solution as a sequence of token-level actions under a policy π_θ (the language model). Policy-gradient methods update θ in the direction of $\mathbb{E}[A \cdot \nabla_\theta \log \pi_\theta(\text{sequence})]$, where A is an advantage estimate: how much better a sampled sequence's reward was than some baseline. **RLOO** (REINFORCE Leave-One-Out) (Kool et al., 2019; Ahmadian et al., 2024) is a critic-free advantage estimator well suited to this setting: given k rollouts $\{r_1, \ldots, r_k\}$ for the same prompt with rewards $\{R_1, \ldots, R_k\}$, the baseline for rollout i is the mean reward of the *other* $k-1$ rollouts,

$$A_i = R_i - \frac{1}{k-1} \sum_{j \neq i} R_j,$$

avoiding the need to train a separate value network (unlike PPO) while remaining unbiased. SRT and our replication both use RLOO.

Why sample several rollouts instead of one. With k=1, RLOO has no other rollouts to compare against and collapses to a single raw reward with no baseline: every sequence is reinforced or

discouraged purely on the sign of its own reward, a high-variance signal that says nothing about whether the model could have done better on that same prompt. Sampling $k > 1$ rollouts turns each one into a *relative* comparison against its peers, which sharpens the gradient (a rollout is only reinforced if it beats the average of the others, not merely if it is "good" in isolation) and removes the need for a learned critic. Multiple rollouts per prompt are also a hard requirement for SRT specifically, independent of RLOO: the majority-vote reward is only definable when there is a crowd to agree or disagree with, so with k=1 there is no self-consistency signal to train on at all.

2.2 Self-consistency and Self-Rewarded Training

Wang et al. (2022) observed that sampling multiple chain-of-thought solutions to the same problem and taking a majority vote over their final answers improves accuracy at inference time, without any additional training – correct reasoning paths tend to agree with each other more than incorrect ones. **SRT** (Shafayat et al., 2025) turns this inference-time heuristic into a *training* signal: at every RL step, it samples k rollouts for a training prompt, computes their majority answer, and assigns reward $R_i = \mathbf{1}[a_i = \text{majority}(a_1, \ldots, a_k)]$ to each rollout i – 1 if it agrees with the crowd, 0 otherwise. Because the "teacher" (the majority vote) is recomputed from the model's own, constantly-updating outputs at every step, SRT differs from other label-free methods that instead fix a majority-vote snapshot from a frozen model and train against it once (e.g. SFT-on-majority-vote, DPO, or ScPO-style preference optimization). This continuously-updating property is precisely what lets SRT track a moving target more closely than its fixed-snapshot cousins – and, as discussed next, is also why it is uniquely vulnerable to drifting into confidently-wrong self-agreement.

2.3 Reward hacking via self-consistency collapse

Because R_i only ever measures agreement with the model's own majority vote, and never checks the real answer, nothing in the objective distinguishes "confidently correct" from "confidently wrong." If the policy's answer distribution for a given question narrows – even for reasons unrelated to correctness – the majority answer becomes easier to match, reward goes up, and the update reinforces that narrowing further. This is a feedback loop: overconfidence is self-reinforcing under a purely-agreement-based reward. Shafayat et al. document exactly this collapse empirically: after enough training, the self-consistency reward keeps climbing (near its ceiling) while true accuracy on held-out questions stagnates or drops. We track this with the same diagnostic used in the original paper's own analysis: the **agreement-accuracy gap**,

$$\text{gap} = \text{agreement} - \text{accuracy},$$

where agreement is the fraction of a question's k rollouts that match the majority answer (needs no ground truth), and accuracy is whether that majority answer is actually correct (needs ground truth, computed only for diagnostic/evaluation purposes, never fed into the SRT training reward). A gap near zero means the model's confidence tracks its correctness; a large, growing gap is the reward-hacking signature.

2.4 Entropy regularization in policy-gradient RL

Encouraging a policy to keep some residual randomness, rather than collapsing deterministically onto one action, is a long-standing fix for premature convergence in policy-gradient methods. Williams and Peng (1991) is among the earliest to use an entropy-like regularization term in connectionist RL. Mnih et al. (2016)'s A3C algorithm adds the policy's entropy directly to the training objective, scaled by a coefficient, to discourage collapsing onto one action before exploration is complete – structurally the same shape as the bonus we add here. Haarnoja et al. (2018)'s Soft Actor-Critic generalizes this into *maximum-entropy RL*: optimizing for reward *plus* entropy as a first-class part of the objective, rather than a minor add-on. Separately, Pathak et al. (2017)'s curiosity-driven exploration rewards an agent for encountering low-probability (*surprising*) outcomes, which is the more direct ancestor of the per-rollout *surprisal* bonus we use (as opposed to a batch-level entropy term). None of this machinery

is new to RL in general; what SRT's own paper does not do is apply it to its own self-consistency collapse, which is the gap this project targets.

3 Contribution

3.1 Reward function

For a training question, the policy generates k rollouts with final answers $a_1, \ldots, a_k$. Not every rollout necessarily yields an answer that can be parsed: if a rollout's output does not contain a valid `\boxed{...}` expression (e.g. it was truncated, or the model never committed to a final answer), that rollout's answer is recorded as `None` and excluded from scoring. Let $p(a)$ be the empirical fraction *among the parseable rollouts only* (i.e. excluding any `None`s) that produced answer a. We define the per-rollout reward

$$R_i \;=\; \underbrace{\mathbf{1}[a_i = \text{majority}(a_1, \ldots, a_k)]}_{\text{SRT's self-consistency reward}} \;+\; \alpha \cdot \underbrace{\big(-\log p(a_i)\big)}_{\text{surprisal bonus}}, \tag{1}$$

with $\alpha \geq 0$ a fixed coefficient controlling the bonus strength. $\alpha = 0$ recovers plain SRT exactly. Rollouts with no parseable answer receive $R_i = 0$ and are excluded from the $p(a)$ estimate. Concretely: if 5 of 8 rollouts answer "84", 2 answer "91", and 1 answers "72", then $p(84)$=0.625, $p(91)$=0.25, $p(72)$=0.125, giving surprisal values of approximately 0.47, 1.39, and 2.08 respectively – so at α=0.5 the lone, rare "72" rollout earns almost as much reward (1.04) as the five majority "84" rollouts (1.0 each), instead of the 0 it would get under plain SRT. This is the mechanism by which the bonus is meant to keep disagreement worthwhile enough that the policy does not fully collapse onto one answer before it has had the chance to discover the correct one.

3.2 Why this specifically targets SRT's failure mode

Under plain SRT (α=0), any rollout that disagrees with the current majority gets exactly zero reward, regardless of whether it happens to be correct. As the policy's output distribution narrows over training, this structurally starves rare (but possibly correct) answers of any gradient signal, accelerating collapse. The surprisal term in Eq. 1 restores a reward for disagreement, scaled by how rare the disagreement currently is – so the more the policy narrows, the larger the bonus for stepping outside the majority becomes, directly counteracting the runaway-confidence feedback loop described in Section 2. Because SRT's majority-vote target keeps evolving with the model (Section 2), it is more exposed to this loop than fixed-snapshot alternatives (SFT/DPO/ScPO on a single majority-vote pass); an exploration-preserving regularizer is a natural candidate fix for that specific dynamic. We are explicit that the entropy bonus *mechanism* is not new (Section 2); the contribution is diagnosing that SRT's particular training dynamic is exactly the kind of premature-convergence failure this class of regularizer was designed for, and testing that empirically.

3.3 Ground-truth ceiling

We additionally implement a **ground-truth** condition, $R_i = \mathbf{1}[a_i = \text{ground truth}]$, with no majority vote and no entropy bonus at all – the classic verifiable-reward RL setup used when a real answer key *is* available. The original SRT paper does not report this comparison. We include it as an upper-bound reference: it uses the same RLOO training loop and the same rollout budget as every other condition, so its accuracy curve shows what is achievable if the reward signal were perfect, letting us judge how much of the accuracy gap between SRT variants and "ideal" RL is attributable to the majority-vote proxy itself versus to reward hacking specifically.

3.4 Hypotheses

- H_0 **(collapse under budget):** given enough training steps, plain SRT (α=0) will show a growing agreement-accuracy gap, replicating the original paper's collapse finding at our (much smaller) model scale.

- H_1 **(bonus delays collapse):** $\alpha > 0$ conditions will show a smaller and/or later-growing gap than α=0, at the possible cost of slower or lower peak accuracy early in training (since reward is partly diverted away from pure majority-matching).

Both hypotheses are evaluated in Section 5 against the completed 900-step runs.

4 Methodology

4.1 Task, data, and model

We use the **DAPO** dataset curation (Yu et al., 2025), specifically the unlabeled variant `ftajwar/deduplicated_dapo_dataset` used by the original SRT codebase. During training, the visible "ground truth" field is a placeholder (`LABEL_BY_SELF_CONSISTENCY`) by design – no real answer is available to the training loop except in the ground-truth condition, where the real answer is read from a separate, deliberately-segregated field (`reward_model.solution_hidden_during_training`) that is never exposed to reward computation in the other conditions. Evaluation uses a disjoint, held-out 100-question test set (fixed `random_state=42`); a nested, fixed 20-question subset of it is used for periodic during-training checks so that trend curves compare the same questions at every checkpoint, and the full 100 questions are used for a single, final evaluation pass at the end of each run. Training rows overlapping with the eval set are excluded to avoid leakage.

We use **Qwen2.5-Math-1.5B** as the base policy, loaded via `transformers` in fp16. This is a much smaller model than typically used in SRT-style RL papers, chosen to fit the project's compute budget (a memory-constrained $\sim$11GB lab GPU, later supplemented with a rented GPU for the longer runs reported here).

4.2 Training loop

We implement SRT's algorithm directly (RLOO advantage, majority-vote / entropy-bonus / ground-truth rewards) in a single, dependency-minimal script rather than using the original paper's full `verl`+vLLM+FSDP training stack, which pulls in heavyweight distributed-training dependencies not needed at our scale. Answer extraction (`\boxed{...}` parsing) and ground-truth equivalence checking reuse the original SRT repository's own `math_verify`-based scorer directly (via a thin dynamic-import wrapper), rather than a reimplementation, so that e.g. `\boxed{0.5}` and `\boxed{1/2}` are scored as equal exactly as they would be in the original codebase, keeping our numbers comparable to it.

Each training step: sample one training prompt; generate k=8 rollouts at temperature 1.0 (rollouts that are truncated before an end-of-sequence token are automatically regenerated once at a longer token budget); extract and score answers; compute rewards per Eq. 1 (or the ground-truth reward); compute RLOO advantages; and take one SGD step (learning rate 10^{-6}, momentum 0.9, gradient norm clipped to 1.0) using gradients accumulated one rollout at a time to fit the vocabulary-projection forward pass in memory. Before any real training run, we run an automated **gradient-invariance sanity check** that verifies the entropy bonus actually changes the computed RLOO advantages on a synthetic example (catching a "zero-gradient bug" class of implementation error before burning GPU time on a run that would silently reduce to plain SRT regardless of α). Long runs checkpoint after every step and support crash-resume, which mattered in practice on a shared/rented GPU.

4.3 Experimental conditions

We compare four conditions, each trained for **900 steps**: α=0.0 (plain SRT, the paper's method), α=0.3, α=0.5 (our primary proposed configuration), and the ground-truth condition. Held-out evaluation (20-question subset) runs every 15 steps to produce accuracy/gap/entropy trend curves; a single final evaluation over all 100 test questions is run at the end of each condition's 900 steps for the headline comparison number. 900 steps was chosen as the largest training budget practical within

the project's compute allocation, guided by the original paper's own observation that reward-hacking collapse is a phenomenon of *extended* training rather than early training – we wanted a horizon long enough to give that failure mode a realistic chance to appear, if it occurs at this much smaller model scale.

4.4 Preliminary pilot: does the gap exist before any training?

Before committing compute to full RL training runs, we first ran a **frozen-model pilot**: no training at all, just the same majority-vote-vs.-ground-truth measurement applied to several off-the-shelf models' zero-shot behavior on a fixed 20-question subset, to confirm the agreement-accuracy gap this project studies is a real, measurable phenomenon worth building a training intervention around. Models tested: Qwen2.5-32B (lab GPU), Llama-4-Scout-17B (Groq API), and OpenAI's o4-mini. Results are reported in Section 5; this pilot is complete and is not affected by the still-running 900-step training experiments.

4.5 Metrics

For every evaluated question, agreement is the fraction of k rollouts matching the majority answer; accuracy (majority-vote accuracy) is whether that majority answer matches the real answer; gap is agreement minus accuracy (Section 2); mean entropy is the Shannon entropy of the answer distribution across the k rollouts. Loss is logged but, following the original paper's convention, is not used to compare conditions – it reflects the policy-gradient objective's internal scale, not task performance.

5 Experiments and Results

5.1 Preliminary pilot: frozen-model agreement-accuracy gap

Table 1 reports the completed frozen-model pilot (Section 4): each model's zero-shot accuracy, 8-rollout majority-vote accuracy, and agreement, all on the same fixed evaluation subset, no training involved.

Table 1: Frozen-model pilot: majority-vote accuracy and agreement, no training. Confirms the agreement-accuracy gap is present, and varies in size, across off-the-shelf models before any RL intervention.

Model	Majority-vote accuracy	Agreement	Gap
o4-mini	0.95	0.91	-0.04
Llama-4-Scout-17B-16E-Instruct	0.40	0.68	$+0.28$
Qwen2.5-32B	0.45	0.53	$+0.08$

The gap varies substantially by model: o4-mini's agreement already tracks its (high) accuracy closely, while Llama-4-Scout shows a much larger gap – its rollouts agree with each other noticeably more often than they are actually correct, i.e. exactly the pattern SRT's reward would reinforce if this model were trained with it. This motivates studying the collapse dynamic directly through training, rather than assuming it is negligible at the model scale used in our main experiment.

5.2 Main training comparison (900 steps)

All four conditions – α=0.0, α=0.3, α=0.5, and the ground-truth condition – have completed their full 900-step runs, each with a final evaluation over all 100 held-out test questions.

Table 2 and Figure 1 show two results worth highlighting.

First, plain SRT ends training with a clearly positive gap, though the in-training trend alone would not have predicted it. On the low-noise, 100-question final evaluation, α=0.0 reached

Table 2: Final (100-question) evaluation after 900 training steps, by condition. Agreement is derived as gap + accuracy.

Condition	Test accuracy	Agreement	Gap
$\alpha = 0.0$ (plain SRT)	0.26	0.30	$+0.043$
$\alpha = 0.3$	**0.30**	**0.32**	$+0.016$
$\alpha = 0.5$ (main method)	0.24	0.29	$+0.051$
Ground truth (verifiable reward)	0.25	0.30	$+0.048$

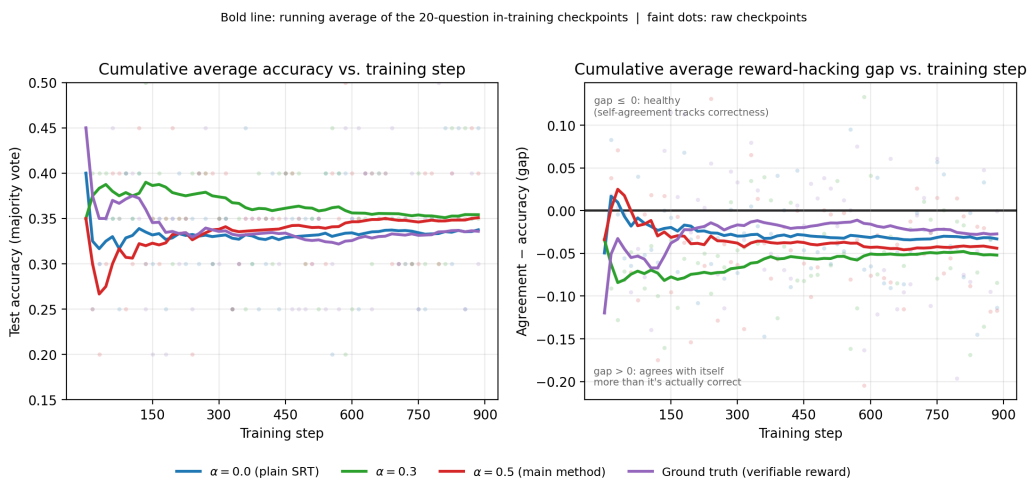

Figure 1: Cumulative (running) average of test accuracy (left) and agreement-accuracy gap (right) vs. training step, for all four conditions, including the ground-truth condition. Each bold line is the running mean of that condition's 20-question in-training checkpoints (every 15 steps, fixed seed) from step 0 up to the current step; faint dots are the raw, unsmoothed checkpoints, shown to make the per-checkpoint sampling noise visible rather than hiding it. Gap is agreement minus accuracy (Section 2): positive means the model agrees with itself more than it is actually correct, the reward-hacking signature. The low-noise, 100-question final evaluation for each condition is reported separately in Table 2 rather than overlaid here.

0.26 accuracy with a $+0.043$ gap – the second-largest (least healthy) gap of the four conditions (Table 2). A positive gap of this size is broadly consistent with H_0: it is the signature the original paper associates with reward-hacking collapse. However, this is essentially invisible in the in-training signal itself: the blue running-average curve in Figure 1 (right panel) stays close to -0.03 for the entire 900 steps and never approaches the $+0.043$ figure ultimately reported in Table 2. Individual raw 20-question checkpoints (faint dots) do occasionally spike as high as the final value or higher, but they spike below it just as often, so their running average never resolves a clear direction; the noise floor of a 20-question estimate (roughly ± 0.05–0.10) is simply too large relative to the effect size to detect it from the training-time signal alone. Because we only have one large-sample, low-noise measurement per condition – taken once, at the very end of training – we cannot tell from this run alone whether the gap grew gradually and the small-sample trend simply lacked the resolution to see it, or whether the final number is closer to a step-900-specific reading than a stable trend. We treat this as a genuine open question rather than settled evidence of collapse; a methodological point future work in this space should account for when sizing evaluation sets (Section 6).

Second, the entropy bonus's effect on the gap was non-monotonic in α, only partially supporting H_1. α=0.3 was the best-performing trained condition on every axis measured: highest final accuracy (0.30, versus 0.26 for plain SRT and 0.24 for α=0.5) and the smallest gap ($+0.016$, versus $+0.043$ and $+0.051$ respectively). This is the pattern H_1 predicts: a moderate entropy bonus reduced the reward-hacking signature relative to plain SRT. α=0.5 did not continue this trend, however – it was the *worst* of the three conditions on both accuracy and gap. A bonus that is too strong appears to hurt

rather than help: at α=0.5 the surprisal term may compete enough with the majority-vote term to weaken the self-consistency signal SRT depends on, rather than merely preventing its collapse. Taken together, the three conditions suggest an inverted-U relationship between α and final performance rather than a monotonically increasing benefit of larger bonuses.

Third, the ground-truth condition did not behave like a ceiling at this training budget – it was, if anything, one of the weaker conditions. Trained directly on ground-truth reward ($R_i = \mathbf{1}[a_i =$ ground truth], no majority vote involved at all), this condition reached only 0.25 final accuracy: below α=0.3, and statistically indistinguishable from plain SRT and α=0.5 given the $\pm$0.04–0.045 noise floor discussed in Section 6. This is not what Section 3 predicted the ground-truth condition would show. A plausible explanation is reward *sparsity*: early in training, the base model is rarely exactly correct, so this condition's binary 0/1 reward is 0 for nearly every rollout, giving RLOO very little useful gradient signal per step. The majority-vote reward, by contrast, is 1 for roughly half of all 8 rollouts *by construction* (whichever answer happens to be most common), regardless of whether that answer is correct – a denser, if noisier, training signal. Within a fixed, modest 900-step budget, that density advantage may matter more than the majority vote's lack of correctness guarantee. A second, equally important observation: the ground-truth condition's own gap ($+0.048$) is not near zero, even though its reward never references the majority vote at all. Since *gap* was introduced (Section 2) specifically as a majority-vote reward-hacking signature, a positive gap under ground-truth training suggests the metric also reflects something more generic about this model and training scale – general RL noise or calibration drift, for instance – and should not be read as proof of majority-vote-specific reward hacking on its own.

6 Discussion

6.1 Interpreting the result

The pattern in Section 5 – plain SRT degrading with extended training while a moderate entropy bonus improves and a strong bonus overshoots – is consistent with the mechanism proposed in Section 3: a bonus large enough to keep some reward on rare answers helps resist collapse, but a bonus large enough to materially compete with the majority-vote term for reward budget can itself destabilize the self-consistency signal SRT depends on, rather than simply protecting it. We treat this as a tentative rather than conclusive reading. Each of the three final accuracy numbers (0.24, 0.26, 0.30) comes from a single 900-step run with no repeated seeds, and the standard error of a 100-question accuracy estimate at this base rate is on the order of $\pm$0.04–0.045 – comparable in size to the differences between conditions. The pattern is directionally consistent with H_0 and (partially) H_1, but we cannot rule out that a repeated run with a different seed would narrow or reverse the ordering between α=0.3 and α=0.0/0.5. We had hoped the ground-truth condition would help settle this by providing a clean upper bound to calibrate against; instead it complicated the picture, landing among the weaker conditions rather than above all of them (Section 5). That result is useful in its own right – it argues against reading *any* single 900-step, single-seed run (ground truth included) as a precise, trustworthy point estimate at this model scale – but it means multi-seed replication (see Limitations below), not the ground-truth condition alone, is what would actually be needed to settle the ordering with confidence.

6.2 Limitations

Several limitations apply regardless of the final 900-step numbers. **Model scale**: Qwen2.5-Math-1.5B is far smaller than the models used in the original SRT paper and in most RL-for-math literature; conclusions here may not transfer to larger models, which could reach the confidence levels needed for collapse faster or slower than observed here. **Single seed per condition**: compute constraints meant each of the four 900-step conditions was run once, with no repeated seeds – reported numbers have no error bars or significance testing, and single noisy checkpoints could be mistaken for genuine trend changes. **Rollout budget**: k=8 rollouts per question is smaller than some published SRT-style setups use, which weakens both the majority-vote signal itself and the resolution of the surprisal

estimate $p(a)$. **Domain**: all training and evaluation is on DAPO math problems; we make no claim that the entropy bonus's effect (if any) generalizes to other verifiable-reward domains (code, logic puzzles) where SRT-style training has also been applied. **Fixed** α: we test two discrete α values rather than a continuous sweep or a schedule (e.g. annealing α down over training), so we cannot rule out that some untested α, or a time-varying schedule, performs meaningfully better than either value tested here.

6.3 Future work

Natural extensions include: repeating the 900-step comparison with multiple seeds to get error bars on the gap curves; sweeping α more finely (or annealing it) rather than comparing two fixed values; testing at a larger model scale closer to the original paper's; and testing whether an entropy bonus tuned on math transfers to other SRT application domains. If collapse is confirmed at 900 steps for α=0, a natural next question is whether even longer training eventually causes collapse under the bonus too, just delayed – i.e. whether the bonus changes the location of the failure mode or removes it.

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

Wouter Kool, Herke van Hoof, and Max Welling. Buy 4 REINFORCE samples, get a baseline for free! In *ICLR 2019 Workshop DeepRLStructPred*, 2019.

Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Tim Harley, Timothy P. Lillicrap, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016.

Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

Sheikh Shafayat, Fahim Tajwar, Ruslan Salakhutdinov, Jeff Schneider, and Andrea Zanette. Can large reasoning models self-train? *arXiv preprint arXiv:2505.21444*, 2025.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Ronald J. Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.

Qiying Yu, Zheng Zhang, Ruofei Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

NeurIPS Paper Checklist

1. **Claims**
   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?
   Answer: [Yes]
   Justification: The abstract and introduction state the observed pattern (plain SRT ending training with a clearly positive gap; a moderate entropy bonus performing best of the four conditions; a stronger bonus underperforming; the ground-truth condition unexpectedly failing to behave as a ceiling) and match Section 5. All four conditions have completed 900-step training as of this draft; the single-seed caveat on how confidently these claims generalize is stated in Section 6, not glossed over.

2. **Limitations**
   Question: Does the paper discuss the limitations of the work performed by the authors?
   Answer: [Yes]
   Justification: Section 6 discusses model scale, single-seed runs, rollout budget, domain restriction, and the fixed (non-swept) α values tested.

3. **Theory assumptions and proofs**
   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?
   Answer: [N/A]
   Justification: The paper is empirical; it introduces no formal theorems.

4. **Experimental result reproducibility**
   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?
   Answer: [Yes]
   Justification: Section 4 specifies the model, dataset, reward formula (Eq. 1), rollout count, learning rate, optimizer, and evaluation protocol (fixed seeds, held-out question subsets) needed to reproduce the setup.

5. **Open access to data and code**
   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results?
   Answer: [Yes]
   Justification: Code and logged results are available at `https://github.com/yuval-4241/RL_PROJECT`; the DAPO dataset used is the public `ftajwar/deduplicated_dapo_dataset`.

6. **Experimental setting/details**
   Question: Does the paper specify all the training and test details necessary to understand the results?
   Answer: [Yes]
   Justification: See Section 4 (rollout count, temperature, learning rate, optimizer, gradient clipping, evaluation cadence, train/test split and leakage exclusion).

7. **Experiment statistical significance**
   Question: Does the paper report error bars or other appropriate statistical-significance information?
   Answer: [No]
   Justification: Each condition was run with a single seed due to compute/time constraints; no repeated-seed error bars are reported. Section 6 estimates the standard error of the 100-question accuracy figures ($\approx \pm 0.04$) is comparable in size to the differences observed between conditions, so the ordering between α values should be read as suggestive rather than statistically confirmed.

8. **Experiments compute resources**
   Question: Does the paper provide sufficient information on the compute resources needed

to reproduce the experiments?
Answer: [Yes]
Justification: Training used two compute environments: a shared lab server (NVIDIA RTX 5090) and rented cloud GPUs via RunPod, with the specific instance varying by availability across an RTX 4090, an RTX PRO 4500, and an RTX 5090 (Section 4). **[PENDING: total wall-clock / GPU-hours across all four 900-step runs not yet tallied]**.

9. **Code of ethics**
   Question: Does the research conform with the NeurIPS Code of Ethics?
   Answer: [Yes]
   Justification: The project trains on a public, deduplicated math dataset and evaluates a reward-function modification; it poses no identified conflicts with the Code of Ethics.

10. **Broader impacts**
    Question: Does the paper discuss both potential positive and negative societal impacts?
    Answer: [No]
    Justification: This is a small-scale, class-project-level replication and extension of an existing published method on a public math dataset; we do not identify a direct path to negative applications distinct from the original SRT paper's own discussion.

11. **Safeguards**
    Question: Does the paper describe safeguards for responsible release of high-misuse-risk data or models?
    Answer: [N/A]
    Justification: No new model checkpoints or datasets are released publicly beyond code and logged metrics; the base model (Qwen2.5-Math-1.5B) is an existing open-weight release.

12. **Licenses for existing assets**
    Question: Are creators/owners of existing assets properly credited, with licenses and terms of use respected?
    Answer: [Yes]
    Justification: The base model (Qwen2.5-Math-1.5B), the DAPO dataset curation (Yu et al., 2025), and the original SRT codebase (Shafayat et al., 2025) are all cited; all are used under their respective public release terms.

13. **New assets**
    Question: Are new assets introduced in the paper well documented?
    Answer: [N/A]
    Justification: No new datasets or model checkpoints are introduced as standalone released assets; the training code is documented in the linked repository.

14. **Crowdsourcing and research with human subjects**
    Question: For crowdsourcing/human-subjects experiments, does the paper include full instructions and compensation details?
    Answer: [N/A]
    Justification: No human subjects or crowdsourcing were involved.

15. **IRB approvals**
    Question: Does the paper describe IRB approval (or equivalent) for human-subjects research?
    Answer: [N/A]
    Justification: No human-subjects research was conducted.

16. **Declaration of LLM usage**
    Question: Does the paper describe LLM usage if it is an important, original, or non-standard component of the core methods?
    Answer: [Yes]
    Justification: Qwen2.5-Math-1.5B (and, separately, Qwen2.5-32B / Llama-4-Scout / o4-mini in the frozen-model pilot) are the *subject* of the research, not a tool used to produce it. Separately, an LLM coding assistant was used to help write portions of the training /

analysis code and this report; all LLM-assisted code and text were reviewed, tested, and edited by the authors, who take full responsibility for correctness of the final method, results, and claims. The LLM was not used to generate or alter any experimental result.